Handoff 12: Admin and tester roles (FB-01)

Date: 2026-10-26 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request, not merged, not deployed. **Migration 0007** (additive). There is no admin screen; the API has two admin routes. No secret is in the repository.

> **How to use this file:** upload it with `build.md`, `docs/plan/feedback-security.md` (section 1) and handoff 07 (server-side sessions). The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy FB-01: an admin role decided only by the
ADMIN_EMAILS environment variable (verified Google emails), shorter admin
sessions (8 hours, or 1 hour idle), and a tester flag on student accounts
that only an admin can set. I'm attaching build.md, the feedback security
plan, handoff 07 and this handoff. Check that no route under /api/admin
can be reached by a student, a spoofed header or an edited cookie; that
an unverified or merely matching email never makes an admin; that the
admin list never appears in a response or log; and that migration 0007
is safe during a deploy. Explain at a first-year computer engineering
level.
```

1. Step 1: was there earlier admin work?

Checked before writing any code: - `git fetch --all`; all local and remote branches; stashes; the working tree; worktrees - `gh pr list --state all` - `git log --all --grep` for admin, tester, FB-01, ENG-04 and "feedback engine" - file names and code on every branch, for handoffs about admin or ENG-04, `ADMIN_EMAILS`, `is_admin`, a tester flag or a migration 0007

Topic	Found
Admin and testers	Only plans: the stage 5 documents (PR #10, commits `a8a0181` and `6aef0b2`, the feedback threat model) and the backlog rows FB-01 to FB-09. **No code, branch, stash, handoff or PR.** So this PR builds it
ENG-04 (feedback engine)	Only the specification `docs/plan/lesson-feedback.md` (PR #15) and its backlog row. **No code, branch or handoff**

Topic	Found
Uncommitted work on this machine	None: only `.DS_Store` files and Next.js's generated `frontend/AGENTS.md`, all untracked

2. Handoff record (build.md template)

Field	Value
Task	FB-01: admin identity from `ADMIN_EMAILS` (verified Google emails only), admin sessions of 8 hours or 1 hour idle, a tester flag on students that only an admin sets, a guarded admin API, tests, docs
Human goal	The owner can be the admin and mark student testers, without any route an attacker or a student could use to become admin
Relevant build.md sections	11 (product foundation), 12 (per-feature gate: real account boundaries tested); `feedback-security.md` section 1 (admin takeover)
Repository and branch	`torontojohn/python-academy`, branch `feat/admin-role`, from `origin/main`
Base commit	`27ab31b` (`main` after PR #16)
Implementer / reviewer	Claude Code / owner, plus ChatGPT or Claude chat
Deployment status	Not deployed; not merged. Needs `ADMIN_EMAILS` set by the owner in Render (section 9)
New dependencies	None

Files changed

File	Change
`backend/app/admin.py`	New. `admin_emails()` (reads and normalizes the variable), `admin_at_sign_in()` (verified email and listed), `is_admin()` and `is_tester()`, the `require_admin` guard, and the routes `GET /api/admin/students` and `POST /api/admin/testers`, registered by `register(app)`
`backend/app/sessions.py`	`start(..., admin=False)`; admin sessions expire after `ADMIN_ABSOLUTE` (8 hours) or `ADMIN_IDLE` (1 hour); new `admin_session(request)`. Learner lifetimes unchanged

File	Change
`backend/app/db.py`	`User.tester` and `UserSession.admin`, both false by default
`backend/migrations/versions/0007_admin_and_testers.py`	New: adds the two columns; the downgrade removes them
`backend/app/main.py`	The Google callback starts an admin session when `admin_at_sign_in` says so; `/api/me` also returns `admin` and `tester`; admin routes registered
`backend/app/limits.py`	A new `admin` rule (60 a minute) for the admin write route; existing limits unchanged
`backend/tests/test_admin.py`	New: 24 tests (section 6)
`backend/tests/test_migrations.py`	Head 0007; a new test that 0007 upgrades and downgrades with data kept
`backend/tests/test_sessions.py`	The route list includes the two admin routes, and a new test that every route is visible to these safety checks (section 4)
`render.yaml`	`ADMIN_EMAILS` with `sync: false`: listed, value never in the file, never overwritten by the Blueprint
`DEPLOYMENT.md`	New section "Admins and testers"
`docs/plan/course-catalog.md`, `docs/plan/feedback-security.md`	Decision 14 (how the admin is identified) recorded as decided, which is part of FB-01

3. Decisions and why

The design defaults were decided by the owner in the task. The choices made while building them:

Decision	Options considered	Reason
Decide admin **at sign-in**, from Google's answer, and store it on the session row; re-check `ADMIN_EMAILS` on every request	Compare the stored account email with the list on each request; store an admin flag on the user	Only Google's answer at sign-in proves the email is verified *now*. The stored email is from account creation. Re-checking the list means removing an email ends access at once. Nothing in the cookie says "admin"
Treat `email_verified` as verified only if it is exactly `true`	Any truthy value	A string `"true"` passes the existing sign-in check, but is not Google's verified flag (tested)

Decision	Options considered	Reason
Admin is separate from the account role (an admin can be a parent account)	An "admin" role	Roles control family features; the owner's account is a parent. No endpoint can make anyone admin
A tester flag only on student accounts; an admin counts as a tester	Testers of any role	Decision 23
Admin actions recorded in the **admin's own** account history as `tester_set` or `tester_cleared` with `user <id>`	Writing the email; writing to the student's history	The owner's default: by user ID, never by email
API only, no admin page	A minimal page	Allowed by the task, and the smallest attack surface. The owner uses two `curl` or console commands (section 9)
Admin routes are added to the app directly, each with the guard, not through an `APIRouter`	An included router with one shared guard	**Found while building** (section 4): this FastAPI version hides included routers from the route lists that the safety tests read. A test now checks every `/api/admin` route has the guard
`ADMIN_EMAILS` documented in `render.yaml` with `sync: false`	Not listing it; putting a value there	Shows it is needed, without ever committing it; the Blueprint never overwrites the dashboard value
A rate limit on the admin write route	No limit	The existing rule, "every write route has a limit", is tested
Migration 0007: two boolean columns, `NOT NULL` with a server default of false	Nullable columns	The old app version keeps working during the deploy, every existing row gets `false`, and the downgrade removes them cleanly (tested on SQLite and PostgreSQL)
`admin` import on its own line in `main.py`	On the same line as the other imports	PR #17 changes that line; the change keeps both PRs mergeable (checked)

4. Bugs found and how they were fixed

- **Near-miss: included routers are invisible to our safety tests.** The first version put the admin routes in an `APIRouter`, included with `app.include_router`. All existing tests passed, which was suspicious: the route-list test should have failed on two unlisted routes.
- Investigating showed FastAPI 0.162 keeps an included router as one opaque `_IncludedRouter` entry in `app.routes`.
- So the route-list test (sessions), "every write route has a limit" (rate limits) and PR #17's leak scan **would all have silently skipped every admin route**.

- Fix: register the routes on the app, each with `require_admin`. New test: `test_no_route_is_hidden_from_these_checks` fails if any route type other than a plain route appears in `app.routes`. Found by: noticing a test that should have failed did not.
- **Noted, not changed: malformed JSON is answered before the guard.** A request with unparseable JSON gets `422` before `require_admin` runs, because FastAPI parses the body first. It reveals nothing (no data, no admin status). Well-formed requests from non-admins always get 403 (tested). Found by a slip in my local `curl` script.

5. Step 1 findings that affect other work

- **ENG-04** has only its specification; building it is still to do.
- PR #20 (backlog sync) listed "the admin handoff" as not found; this PR is that work. Its backlog rows are in section 8.

6. How it was tested

Kind of test	What it proves
`ADMIN_EMAILS` parsing (6 cases)	Missing, empty, only commas, spaces, mixed case and duplicates are handled; empty means no admins
Sign-in through the real Google callback (only Google's answer replaced)	A verified, listed email (mixed case on both sides) gets an admin session expiring after exactly 8 hours, `/api/me` says `admin: true, tester: true`, and admin routes work. An unverified email (`false`, missing, or the string `"true"`) is never admin. An unlisted email gets a normal 7-day session
Revocation	Removing the email from `ADMIN_EMAILS` gives 403 on the next request
An email match alone	The owner's account with a session not started as admin gets 403
401 and 403 for every admin route (found by listing the app's routes, not a hand list)	Signed out 401; pending, parent, student and tester-student accounts 403; spoofed headers (`X-Admin`, `X-Forwarded-Email`, `X-Forwarded-User`) 403; a cookie edited to add `admin: true`, a user ID and the owner's email 403; the admin 200 on every route
Every admin route has the guard	Structural check of each route's dependencies
Admin session lifetimes (fake clock)	Used every 30 minutes, still refused at exactly 8 hours; refused after 1 hour unused; learner sessions still last past 23 hours
Tester flag	The admin sets and clears it; non-students get 422; unknown IDs 404; history rows hold the admin's ID and `user <id>`, no email; **a student cannot set their own flag (403, flag unchanged)**; a tester student sees `tester: true`, `admin: false`
The admin list stays secret	With two listed emails, the one who never signs in appears in no response (admin and student views) and no log line; the owner's email appears in no log line

Kind of test	What it proves
Migration 0007 (SQLite and PostgreSQL)	Upgrade from 0006 keeps rows and defaults both flags to false; downgrade removes both columns and keeps rows; upgrade again reaches head

Results:

Command	Result
`cd backend && .venv/bin/python -m pytest -q tests/test_admin.py`	`24 passed`
`cd backend && .venv/bin/python -m pytest -q`	`239 passed, 10 skipped` (the 10 need PostgreSQL)
Same with `TEST_POSTGRES_URL` set to a local throwaway PostgreSQL 16	`249 passed`
`cd frontend && npx tsc --noEmit`, `npm run build`, `CI=1 npm run test:e2e`	No errors; compiled; `30 passed` (frontend unchanged)
Backend tests on a trial merge with open PR #17	`281 passed, 10 skipped`

Deliberate-break controls (each undone afterwards):

Break	Tests that failed
The 403 guard removed (any signed-in user let through)	6: both routes' 401/403 test, the student-sets-own-flag test, the email-match-alone test, the revocation test, the `"true"`-string test
`email_verified` ignored at sign-in	The unverified-email test
Admin sessions given the learner idle limit	The 1-hour idle test
One admin route registered without the guard	7, including the structural guard test

A real local run (uvicorn on PostgreSQL, `ADMIN_EMAILS` set, sessions made by the same `sessions.start` call the Google callback uses): - A database at 0006, as production is now, logged `Database schema is at migration 0007` after starting this branch. - Signed out: 401. Student: 403 on both routes, including setting their own flag. - Admin: 200, set the student's flag, and the student's `/api/me` then said `tester: true`. - 0 log lines contained an admin email.

Mocks: - Google's answer to the sign-in callback is replaced in tests; the callback route, sessions and cookies are real. - The local run created sessions directly with `sessions.start`, not through Google. - The Playwright tests mock the API, as before; the frontend did not change.

7. Known limitations

- **Adding an email takes effect at the next sign-in,** because a session's admin status is set when it starts. Removing one takes effect at once.
- **No admin page:** the owner uses the API (section 9). A page comes with the inbox (FB-03, FB-06) if wanted.
- **Admin status is not shown anywhere in the app yet,** only in `/api/me`.
- `ADMIN_EMAILS` **must be set by hand** in Render; until it is, nobody is an admin.

8. Backlog changes (not edited, as instructed)

- **FB-01** → `in_review (PR for handoff 12)`, then `verified` after the live checks below.
- **FB-03** ("Admin role and server-side guard, with tests for 403 on every admin route") is largely delivered here, the guard and the route-enumeration tests included. Suggest marking it in_review together with FB-01, or narrowing it to "guard on inbox routes".
- **FB-04** ("Admin page to mark students as testers") has the API but no page.
- **Decision 14** is now answered (recorded in `course-catalog.md` section 9 and `feedback-security.md` section 1); the backlog's decision table still says "proposed".
- Stage row: "12. FB-01 | Admin and tester roles | in_review (handoff 12)".

9. Live checks for you after merging

Merging redeploys the API.

Before merging: in Render → **python-academy-api** → Environment, add `ADMIN_EMAILS` = your own Google email. Save it, which redeploys. Never paste it into a chat.

- [ ] **Migration 0007.** Render → python-academy-api → Logs shows `Database schema is at migration 0007`, and `curl -s https://python-academy-dji8.onrender.com/health` returns `{"status":"ok"}`.
- [ ] **Signed out gets 401:**

`sh curl -s -o /dev/null -w "%{http_code}\n" https://python-academy-dji8.onrender.com/api/admin/students`

prints `401`. - [ ] **A normal student gets 403.** Signed in as a test student, open `https://python-academy-dji8.onrender.com/api/admin/students` in the browser. It shows `{"detail":"This action is unavailable for this account"}`. - [ ] **Your admin account gets in.** Sign out and sign back in with the email in `ADMIN_EMAILS`; admin status starts at sign-in, so this is required. Open the same address: it lists the student accounts (ID, email, tester), and `/api/me` shows `"admin":true`. - [ ] **Set a tester.** Still signed in as admin, in DevTools → Console on the app (replace `2` with a student ID from the list):

`js await (await fetch('/api/admin/testers', {method: 'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({user_id: 2, tester: true})})).json()`

It returns `{id: 2, tester: true}`. "Your account history" then shows "tester set". Sign in as that student: `/api/me` shows `"tester":true`. - [ ] **Admin sessions are short.** Leave the admin browser unused for over an hour, then reload `/api/admin/students`: it gives 401 and you need to sign in again. - [ ] **Logs hold no admin email.** Search the API logs for your email: nothing beyond Render's own request lines, which do not contain it.

10. Next concrete action

The owner sets `ADMIN_EMAILS`, reviews and merges this PR, and runs section 9. Then FB-02 (the anonymous feedback table and its counters), and the inbox (FB-05, FB-06) behind this guard. SEC-09 (enforcing the nonce policy) should come before the inbox ships.